

DEEP REINFORCEMENT LEARNING

365.382 (SS26)

**RL in Modern Foundation Models:
From Chatbots to Generalist Agents**



Sohvi Luukkonen
LIT AI LAB
Institute for Machine Learning

Recap and Motivation

Model-free RL

Offline

No data collection

Offline Imitation Learning

Behavior cloning

Offline RL

Only use offline data
CQL, IQL, DT

Online

Policy data collection

Online Imitation Learning

Dagger

Requires expert data.
Doesn't need rewards.

Off-policy RL

Reuse data from
other policies

Replay buffer

DQN, DDPG,
SAC

Mult. grad step

PPO, Importance
sampling

On-policy RL

Only data from
current policy

REINFORCE

Inspired from Chelsea Finn.

Critic only, Actor-Critic, Actor only

Model-Based RL

Plan with the model

look ahead before acting

Tree search

MCTS, AlphaGo, AlphaZero

Trajectory optimization

random shooting, CEM, MPC

Train with the model

learn from imagined data

Imagined transitions

Dyna, Dyna-Q, MBPO

Latent rollouts

World Models, Dreamer

What counts as a model?

rules / simulator learned dynamics latent world model

Two Trends Reshaping AI

Foundation models

LLMs, vision-language models,
multimodal models

- trained on large-scale data;
- strong general-purpose representations;
- adapted by prompting, SFT, or post-training.

Agentic AI

models that act, use tools,
and interact with environments

- multi-step reasoning and planning;
- tool use, code execution, browsing;
- feedback from users, tests, or environments.

How do RL concepts reappear when foundation models
become decision-making systems?

Autoregressive Generation as Sequential Decision-Making

Language modeling view

$$p_{\theta}(y \mid x) = \prod_{t=1}^T p_{\theta}(y_t \mid x, y_{<t})$$

- predict the next token;
- train by maximum likelihood;
- evaluate by likelihood or downstream quality.

RL view

$$\pi_{\theta}(a_t \mid s_t) = p_{\theta}(y_t \mid x, y_{<t})$$

- policy: stochastic distribution over tokens;
- state: prompt and generated prefix;
- action: next token or tool action;
- reward: preference, verifier, task success.

What Makes Language RL Difficult?

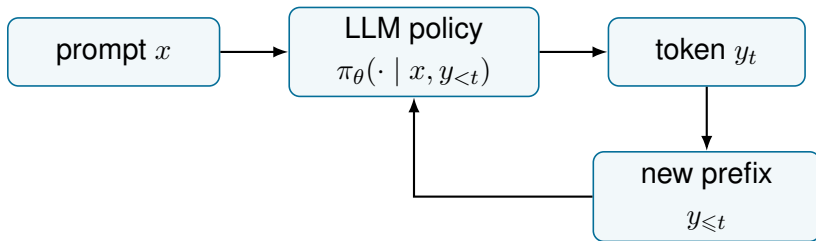
- **Huge discrete action space**: tens or hundreds of thousands of tokens.
- **Long horizons**: early words shape later possibilities.
- **Sparse sequence-level rewards**: the answer is judged after generation.
- **Partial objectives**: helpfulness, truthfulness, style, safety, brevity, and user intent can conflict.
- **Distribution shift**: optimizing the model changes the text people see and rate.

Good news

We already studied the ingredients: behavior cloning, preference learning, policy gradients, KL constraints, reward hacking, and model-based thinking.

LLM Training

LLM as a Policy



- Temperature, top- p , and decoding rules are part of the effective policy.
- The same network can act as a text simulator or as an environment actor.

Pretraining: Behavior Cloning from Text

Objective

$$\max_{\theta} \sum_{(x,y) \in \mathcal{D}_{\text{text}}} \sum_{t=1}^T \log \pi_{\theta}(y_t \mid x, y_{<t})$$

- Imitates the conditional distribution of internet, book, code, and other text.
- This is powerful because the dataset contains many latent skills.
- It is not the same as optimizing for being a helpful assistant.

Why Pretraining Is Not Enough

Pretraining learns

- plausible continuations;
- facts and styles in the data;
- code and reasoning patterns;
- many conflicting behaviors.

Users want

- instruction following;
- calibrated answers;
- refusal in unsafe cases;
- consistency with preferences.

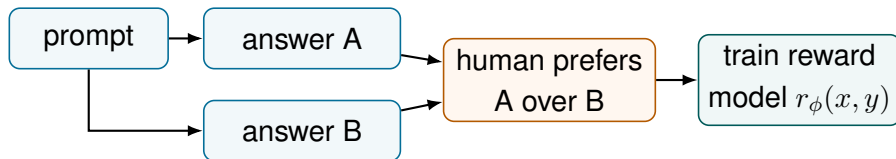
Supervised Finetuning: Instruction-Following Imitation

- Collect prompt-response demonstrations:

$$(x, y^*) \sim \mathcal{D}_{\text{demo}}$$

- Fine-tune with the same token-level likelihood objective.
- Imitates answers written by humans, teachers, or stronger models.
- **Strengths**: simple, stable, cheap relative to RL, and usually the first step toward an assistant.
- **Limitation**: SFT says "copy this behavior", not "search for outputs humans prefer".

From SFT to Preferences



- Humans often find comparison easier than writing the ideal answer.
- Preference data gives a relative signal: $y_w \succ y_l$.
- This is exactly the reward-learning problem from preference-based RL.

Reward Model from Comparisons

Bradley-Terry preference model

$$P_{\phi}(y_w \succ y_l \mid x) = \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

Loss

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))]$$

- The reward model scores complete responses.
- It is a learned proxy for human preference, not ground truth.

RLHF Pipeline

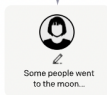
Step 1

**Collect demonstration data,
and train a supervised policy.**

A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.



Step 2

**Collect comparison data,
and train a reward model.**

A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.



The policy
generates
an output.



Once upon a time...



The reward model
calculates a
reward for
the output.

r_k

The reward is
used to update
the policy
using PPO.

Introducing ChatGPT, OpenAI, 2022.

RLHF Objective: Reward with a KL Penalty

KL-regularized objective

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[r_{\phi}(x, y) - \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right]$$

- $r_{\phi}(x, y)$ is the learned reward model.
- π_{ref} is usually the SFT policy.
- The KL penalty keeps the policy close to the reference model.
- This is the target objective; PPO is one way to optimize it approximately.

PPO Clipping for Language Models

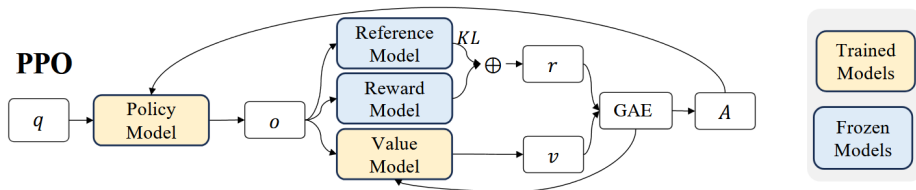
Clipped policy-gradient objective

$$L^{\text{PPO}}(\theta) = \mathbb{E}_{y_t \sim \pi_{\theta_{\text{old}}}(\cdot | x, y_{<t})} [\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

$$\rho_t(\theta) = \frac{\pi_{\theta}(y_t | x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t | x, y_{<t})}$$

- PPO constrains each update relative to the previous policy $\pi_{\theta_{\text{old}}}$.
- The **advantage** A_t is estimated using **rewards** and a learned **value function**.
- In RLHF, the reward often includes the **reward-model score** and a **KL penalty**.

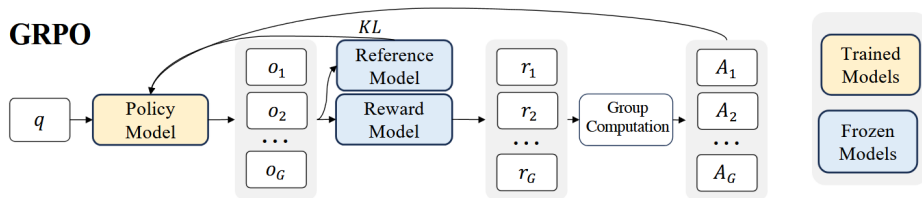
Two Anchors in PPO-Style RLHF



Two different anchors

KL penalty	π_{θ} vs. π_{ref}	stay close to SFT
PPO clipping	π_{θ} vs. $\pi_{\theta_{\text{old}}}$	stable update step

Group Relative Policy Optimization (GRPO)



- Sample a group of outputs for the same prompt.
- Convert rewards into **relative advantages** within the group.
- Apply the same group advantage to all tokens of one output.
- **Avoid a separate value function**, reducing memory and compute.

GRPO Objective

PPO-style update with group-relative advantages

$$L^{\text{GRPO}}(\theta) = \mathbb{E}_{y_1, \dots, y_G \sim \pi_{\theta_{\text{old}}}(\cdot | x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} (C_{i,t}(\theta) - \beta D_{\text{KL},i,t}) \right],$$

$$C_{i,t}(\theta) = \min \left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right),$$

$$\rho_{i,t} = \frac{\pi_{\theta}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | x, y_{i,<t})}, \quad \text{and} \quad \hat{A}_i = \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G}) + \delta}.$$

- G outputs are sampled for the same prompt x .
- $\hat{A}_i$ is sequence-level, but used for all tokens in output y_i .
- $D_{\text{KL},i,t}$ is the token-level penalty to the reference model.

RLHF Failure Modes

- **Reward hacking**: optimize what the reward model likes, not human intent.
- **Overoptimization**: quality can drop after too many RL steps.
- **Preference misspecification**: style or confidence may be rewarded over correctness.
- **Mode collapse**: generic safe-sounding answers become attractive.
- **Distribution shift**: the reward model scores outputs unlike its training data.

Learned reward models are useful, but they are imperfect proxies.
For some tasks, we can replace them with automatic checks.

Reinforcement Learning with Verifiable Rewards (RLVR)

Replace learned preference rewards with **automatic checkers** when the answer can be verified.

- **Math**: final answer matches a verifier.
- **Code**: tests pass.
- **Tool use**: API result satisfies a check.
- **Games**: score or win/loss is observed.
- **Structured domains**: symbolic constraints are satisfied.

Cheap verifier rewards enable many sampled attempts per prompt and have been central to recent **reasoning-model** training and specialized agent development.

RLVR: Reward Design

- A generated answer must be both **checkable** and **correct**.
- If the answer cannot be extracted, correctness cannot be evaluated reliably.
- Common formats: JSON, XML-style tags, or fixed answer fields.
- Format failures can lead to underestimating model capabilities.

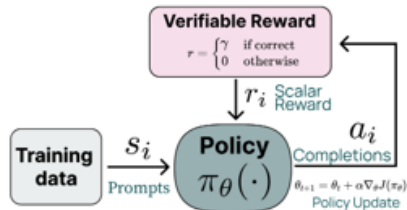
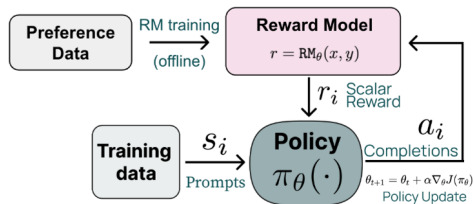
Use a **composite reward** to teach both format and correctness.

$$r(y) = \lambda_{\text{format}} r_{\text{format}}(y) + \lambda_{\text{correct}} r_{\text{correct}}(y)$$

- **Format reward**: answer can be extracted or parsed.
- **Correctness reward**: extracted answer passes the verifier.
- **Optional constraints**: length, tool-use validity, syntax validity.

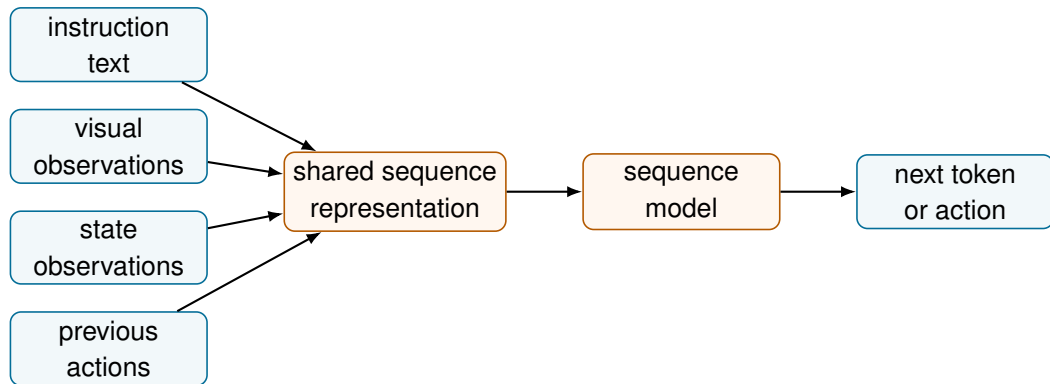
SFT vs. RLHF vs. RLVR

Method	Signal	Strength	Main risk
SFT	demonstrations	stable imitation	limited by examples
RLHF	human preferences	aligns subjective quality	proxy reward hacking
DPO	preference pairs	simple and stable	offline preference limits
RLVR	automatic verifier	scalable objective signal	narrow rewards, hacking verifier



Generalist Agents

Beyond Text: Everything as Sequences



- Tokenization provides a common interface for text, perception, state, and actions.
- Once represented as sequences, language modeling and action prediction become structurally similar.

Gato: One Model, Many Tasks

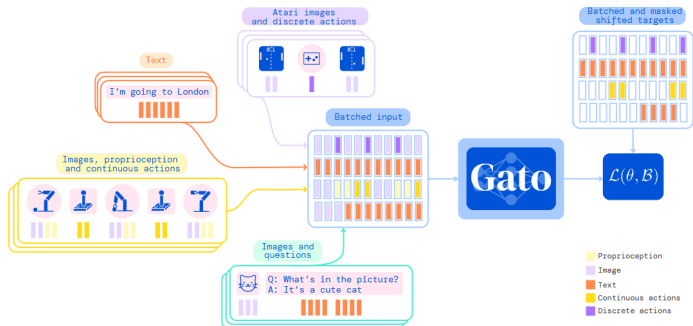
- Gato is a **multi-modal, multi-task, multi-embodiment** generalist policy.
- Same Transformer weights are used for tasks such as:
 - ☐ Atari games;
 - ☐ robot control;
 - ☐ image captioning;
 - ☐ dialogue and text tasks;
 - ☐ simulated navigation.
- Inputs and outputs are **serialized into token sequences**.



Reed et al., 2022.

Gato: A Generalist Policy Trained by Imitation

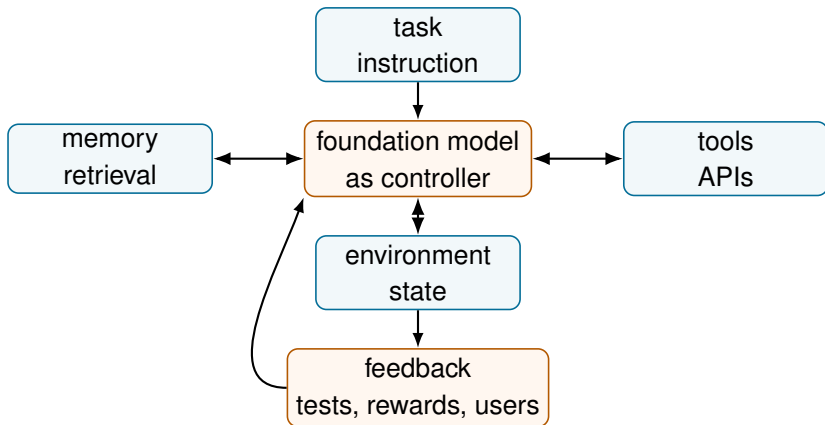
Gato is deployed as a policy over observations and actions, but trained primarily by supervised sequence modeling on mixed datasets.



A policy can be general
without being generally intelligent.

Reed et al., 2022.

From Generalist Policies to Agentic Systems



- Gato learns one policy over tokenized observations and actions.
- Modern agents add tools, memory, retrieval, evaluators, and control loops.
- Learning can target the model, tool use, planning, or task success.

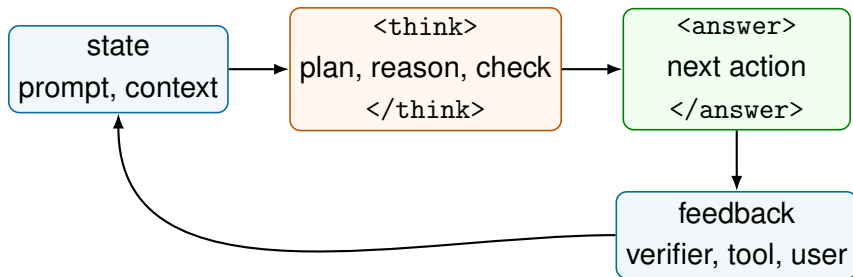
Agent Training Signals

- **Imitation**: copy demonstrations, expert traces, or stronger models.
- **Offline RL**: learn from logs of tool use or interaction.
- **Preference optimization**: prefer better trajectories or answers.
- **Verifiable rewards**: pass tests, solve math, win games, satisfy constraints.
- **Online RL**: learn from live interaction, usually with strong safety filters.

Why the mix matters

Pure online trial-and-error is often too expensive or risky, so practical systems lean heavily on data, simulators, and verifiers.

Planning Before Acting: Text as Computation



- The model can spend tokens computing before emitting the action.
- The final answer, tool call, or command is the action seen by the environment.
- Feedback can trigger another round of planning and replanning.

Classic RL Agents vs. Generalist Agentic Systems

Component	Classic RL agent	Generalist agentic system
Policy	task-specific network	foundation model as controller
State	fixed observation space	context, memory, screen, files, history
Action	fixed action space	tokens, tool calls, clicks, code, commands
Reward	environment reward	tests, users, preferences, task success
Environment	simulator or task MDP	software, web, tools, people, simulators
Training	RL in one task distribution	mixture of SFT, RLHF, RLVR, tools, logs

Takeaway

Generalist agents still fit the RL vocabulary, but the state, action, reward, and environment become broader and less cleanly specified.